

Documentacion de razonamiento y arquitectura

Análisis técnico de la aplicación implementada: arquitectura, contexto autenticado, personalización, validación de planes y seguridad conversacional.

Alcance Aplicación de fitness y nutrición personalizada con planes de 28 días, progreso y chat contextual.	Versión documentada Estado actual del repositorio, incluida la evolución del contexto de perfil y conversación.
Fecha 4 de octubre de 2026	Audiencia Evaluación académica, desarrollo, operación y revisión de seguridad.

Conclusión ejecutiva. La versión actual convierte el perfil autenticado y el plan persistido en la fuente de verdad de la conversación. El asistente responde con datos verificables, distingue restricciones alimentarias próximas y valida un plan antes de reemplazar el vigente.

Este documento describe el comportamiento implementado en el código revisado. No sustituye una auditoría de producción ni afirma que los servicios estén desplegados o activos.

1. Resumen del sistema

FitLife AI es una aplicación web para crear y consultar planes de ejercicio y nutrición. La persona usuaria crea una cuenta, mantiene su perfil, genera un calendario de 28 días, registra progreso y conversa con un asistente limitado al dominio de bienestar.

La actualización documentada refuerza dos garantías: el chat deja de depender del historial visual del navegador y la generación del plan no se acepta si contradice el perfil activo.

Capacidades implementadas

Capacidad	Cómo se resuelve	Fuente de verdad
Identidad	Registro e inicio de sesión; contraseña derivada con scrypt y token firmado HS256.	users y token vigente.
Perfil seguro	Payload cerrado, rangos, normalización de restricciones y reglas de compatibilidad.	profiles validado en backend.
Plan validado	Planificador local crea 28 jornadas y el validador contrasta comida, disponibilidad, capacidad y objetivo.	plans + plan_days + perfil vigente.
Progreso	Marca por fecha y plan; la API calcula siguiente entrenamiento y último completado.	progress + completed_days.
Chat contextual	Contexto central por usuario, router conversacional, memoria compacta y RAG local cuando corresponde.	perfil, plan, progreso, memoria y base local.

Objetivos de diseño

- Mantener aislados los datos de cada cuenta y no usar el historial visual como memoria durable.
- Evitar que el modelo invente ejercicios, comidas, progreso o restricciones de una persona.
- Permitir conversación natural de bajo riesgo: saludo, agradecimiento, ayuda y consultas sobre perfil, plan, rutina, alimentación y progreso.
- Bloquear en el borde solicitudes de inyección, jailbreak, riesgo y temas realmente ajenos antes de agentes, RAG o LLM.

2. Arquitectura de alto nivel

La arquitectura separa presentación, API, dominio, persistencia e IA local. El backend es el punto de control: autentica, carga contexto por cuenta, aplica reglas del perfil, decide la ruta y valida los resultados.

Usuario React + Vite	API FastAPI + JWT	Dominio Reglas + agentes	Datos PostgreSQL	IA local RAG / LLM opcional
--------------------------------	-----------------------------	------------------------------------	----------------------------	---------------------------------------

Principio rector. El perfil y el plan persistidos, filtrados por usuario autenticado, determinan lo que el asistente puede afirmar sobre rutina, comidas, progreso y restricciones.

Responsabilidades por capa

Capa	Responsabilidad	Elementos
Frontend	Captura el perfil, muestra calendario y chat; conserva sólo estado de interfaz y sesión.	React, Vite, AuthContext, Profile, Plan y Chat.
API	Aplica contratos REST, autorización y orquestación de contexto y decisiones.	FastAPI, Pydantic, current_user, db_session y main.py.
Dominio	Genera planes, clasifica intención, resuelve respuestas deterministas y valida compatibilidad.	planner, conversation, router, fitness, nutrition y plan_rules.
Datos	Persiste identidad, perfil, plan, días, progreso, sesiones y memoria por cuenta.	SQLAlchemy + PostgreSQL.
IA local	Recupera conocimiento local; la generación es un apoyo y no una autoridad de datos personales.	LocalVectorStore, LocalRetriever y adaptador LLM.

Despliegue

docker-compose define PostgreSQL, Ollama, una inicialización del modelo, backend y frontend. Los volúmenes separan datos relacionales, modelo de Ollama e índice vectorial. Por defecto, la interfaz se publica en el puerto 5180 y la API en 8010.

3. Contexto autenticado y personalización

La versión actual incorpora un cargador central de contexto: `get_user_context(db, authenticated_user_id)`. En una sola operación lógica carga usuario, perfil, plan activo, días del plan, progreso y memoria compacta exclusivamente para el identificador autenticado.

JWT válido current_user	UserContext perfil + plan + días	Respuesta determinista o asistida	Memoria hecho compacto por cuenta
-----------------------------------	--	---	---

Garantías del contexto

Regla	Comportamiento implementado	Resultado
Aislamiento	Cada consulta usa user_id autenticado para perfil, plan, progreso, sesiones y memoria.	No se mezclan respuestas o estado entre cuentas.
Mínimo necesario	El agente nutricional recibe sólo objetivo y restricciones; respuestas de perfil omiten medidas sensibles si no son necesarias.	Menor exposición de datos personales.
Identidad honesta	Ante "¿Cómo me llamo?", el asistente no inventa un nombre personal; informa el username autenticado si procede.	No fabrica atributos ausentes.
Sin plan	Saludo, ayuda y consultas de perfil siguen disponibles aun sin plan generado.	La experiencia no se degrada a OFF_TOPIC.

Perfil como contrato de personalización

El perfil normaliza objetivo, disponibilidad, horas semanales y restricciones. En particular, **sin lactosa** y **sin lácteos** son restricción distintas: la primera excluye lactosa; la segunda excluye productos lácteos. El frontend expone ambas opciones y el backend conserva su semántica separada.

Regla de coherencia. Si cambia un dato que afecta la planificación - por ejemplo objetivo, disponibilidad, horas o restricciones - el plan queda marcado como **needs_review**. Guardar el mismo perfil no invalida el plan.

4. Generación y validación del plan

El planificador crea 28 días con entrenamientos, recuperación y cuatro tiempos de comida. Antes de persistir la propuesta, `validate_plan_against_profile` la contrasta de forma determinista con el perfil normalizado.

Perfil validado	Planificador 28 días	Validador reglas estructuradas	Persistir sólo si es válido
-----------------	-------------------------	-----------------------------------	--------------------------------

Reglas de validación post-generación

Regla	Evidencia revisada	Decisión
Restricciones alimentarias	Cada comida contiene <code>dietary_properties</code> estructuradas: carne, lácteos, lactosa, huevo, gluten y frutos secos.	Se rechaza cualquier propiedad incompatible; no se decide por palabras del nombre del plato.
Disponibilidad	Cada entrenamiento se compara con el día de semana permitido en el perfil.	No se acepta una sesión fuera de los días seleccionados.
Capacidad	Los minutos de entrenamiento se acumulan por semana frente a <code>horas_semanales</code> x 60.	No se acepta una semana que exceda la capacidad declarada.
Objetivo	El foco del entrenamiento se compara con la categoría del objetivo.	No se acepta un foco discordante.
Integridad	El plan anterior no se elimina antes de que la nueva propuesta pase validación.	Una generación inválida devuelve 422 y conserva el plan válido previo.

Catálogo y propiedades alimentarias

El planificador adapta un catálogo controlado. Las restricciones vegetarianas, veganas, sin gluten, sin frutos secos, sin lactosa y sin lácteos se expresan mediante propiedades booleanas. Esto permite explicar y auditar una incompatibilidad concreta sin inferencias ambiguas.

Interpretación segura. Una comida láctea certificada sin lactosa puede ser compatible con “sin lactosa”, pero no con “sin lácteos”. La respuesta del chat explica esta diferencia y ofrece actualizar el perfil cuando la expectativa de la persona sea más estricta.

5. Flujo de razonamiento del chat

POST /api/v1/chat evalúa guardrails antes de invocar agentes, RAG o LLM. Si la entrada es permitida, la ruta conversacional tiene prioridad para consultas sociales y de resumen; después se resuelven plan, fitness y nutrición con evidencia persistida.

1. Entrada mensaje + sesión	2. Guardrail inyección, riesgo, dominio	3. Contexto cuenta autenticada	4. Intención social, perfil, plan	5. Respuesta datos, RAG o LLM	6. Salida validar y memorizar
--------------------------------	---	-----------------------------------	--------------------------------------	----------------------------------	-------------------------------------

Intenciones conversacionales añadidas

Familia	Ejemplos	Fuente y efecto
Social y ayuda	Hola, gracias, ¿qué puedes hacer?	Respuesta determinista sin requerir plan.
Perfil	Revisa mi perfil, ¿qué restricciones tengo?, ¿cómo me llamo?	Perfil autenticado y username; información mínima necesaria.
Resúmenes	Resume mi plan, rutina, alimentación o progreso.	plan_days y progreso propios; no se inventan métricas.
Auditoría	¿Mi plan respeta mi perfil?, me estás poniendo lácteos.	Validador de plan y perfil actual; explica conflicto, versión o regeneración.
Seguimiento	¿Y de mi plan?, ¿cómo hago ese ejercicio?	Memoria compacta y último ejercicio o tema válido, seguida de RAG si aplica.
Dominio especializado	¿Qué me toca hoy?, ¿cómo preparo esta comida?	Plan persistido primero; RAG y LLM sólo para completar una explicación permitida.

Precedencia de evidencia

La respuesta privilegia datos de menor riesgo: (1) perfil y plan persistidos, (2) progreso y memoria por usuario, (3) conocimiento local/RAG y (4) LLM opcional. Así, una respuesta factual puede seguir funcionando si el proveedor de generación no está disponible.

6. Guardrails, privacidad y límites

Los guardrails no se usan para negar conversaciones cotidianas dentro de FitLife. Distinguen saludo, ayuda, perfil y resúmenes permitidos de solicitudes que quedan fuera del dominio o presentan riesgo. La evaluación de seguridad conserva precedencia sobre un seguimiento contextual.

Controles implementados

Área	Control	Comportamiento
Autenticación	script con salt aleatorio y JWT HS256 con expiración.	No se guardan contraseñas en claro; token inválido o expirado produce 401.
Autorización	current_user y filtros de propietario.	Una cuenta no puede leer plan, progreso, sesiones o memoria de otra.
Entrada IA	Detección de inyección, jailbreak, solicitudes inseguras y OFF_TOPIC real.	Bloqueo antes de agentes, RAG y LLM.
Conversación	Router detallado y clasificación conversacional antes de la extracción de entidades.	Preguntas breves y naturales no se degradan a fuera de dominio.
Salida IA	Filtro de información sensible y política de dominio.	Una respuesta generada que incumpla reglas se sustituye por una controlada.
Privacidad	Contexto por cuenta y divulgación mínima de campos del perfil.	El chat no presenta datos ajenos ni atributos no registrados.

Límites declarados

- FitLife ofrece orientación general de bienestar; no realiza diagnóstico médico ni atención de emergencia.
- La clasificación es explicable y basada principalmente en reglas. Debe ampliarse con vocabulario y pruebas de regresión conforme crezca el dominio.
- El secreto JWT de desarrollo no debe usarse en producción. Un despliegue real requiere secreto fuerte y configuración gestionada.
- La aplicación persiste mensajes y memoria por usuario. Retención, consentimiento, exportación y borrado de datos son extensiones necesarias antes de operar con usuarios reales.

7. Modelo de datos, contratos y operación

SQLAlchemy modela entidades asociadas al usuario. El esquema incorpora una migración ligera e idempotente para **plans.needs_review**, con el fin de distinguir un plan vigente de uno que ya no corresponde a la versión de perfil.

Entidades persistentes

Entidad	Propósito	Integridad
users	Cuenta y hash de contraseña.	username único; id como clave de relaciones.
profiles	Perfil normalizado y versión.	Una fila por usuario; sólo backend valida y persiste.
plans / plan_days	Plan activo, perfil_version, needs_review y 28 jornadas estructuradas.	El plan se consulta y valida en relación con su propietario.
progress	Ejecución por fecha, estado y sello temporal.	Filtrado por user_id y plan_id; evita fechas ajenas al plan.
chat_sessions / chat_messages	Conversación persistida y trazabilidad de intención.	La sesión pertenece al usuario autenticado.
user_memory	Último ejercicio, comida, entrenamiento, intención y tema.	Una memoria compacta por usuario, opcionalmente ligada al plan.

Contratos REST relevantes

Método	Ruta	Propósito
POST	/api/v1/auth/signup y /login	Crea cuenta o inicia sesión y devuelve access_token.
GET / PUT	/api/v1/profile	Lee o actualiza perfil; indica si el plan necesita regenerarse.
POST / GET	/api/v1/plan/generate y /plan	Genera sólo un plan validado o devuelve calendario y needs_review.
POST	/api/v1/progress	Marca o desmarca una fecha perteneciente al plan.
POST / GET	/api/v1/chat y /chat/{session_id}	Envía una pregunta o recupera sólo historial propio.
GET	/api/v1/context	Entrega contexto durable y estado resumido sin restaurar historial visual.

8. Verificación, trazabilidad y evolución

La actualización se verificó con pruebas automatizadas del backend y frontend, construcción de producción del frontend, compilación del backend y revisión de espacios del diff.

Evidencia de calidad

Área	Cobertura comprobada	Resultado
Backend	API, guardrails, aislamiento, perfil, planificador, conversación, memoria y validación de incompatibilidades.	46 pruebas aprobadas.
Frontend	Reglas del perfil y distinción entre sin lactosa y sin lácteos.	12 pruebas aprobadas.
Build	Compilación de producción del frontend y compilación de módulos backend.	Correctas.
Integridad	git diff --check y persistencia de plan anterior ante generación inválida.	Sin errores de formato; regresión cubierta.

Trazabilidad de la versión actual

Tema	Archivos representativos	Hallazgo
Contexto por usuario	backend/app/user_context.py; main.py	Carga centralizada de usuario, perfil, plan, días, progreso y memoria.
Validación de planes	backend/app/validation/plan_rules.py; agents/planner.py	Propiedades alimentarias estructuradas y verificación previa a persistir.
Perfil	validation/profile_rules.py; UserProfileForm.jsx	Restricciones canónicas, incluida la separación sin lactosa / sin lácteos.
Conversación	agents/conversation.py; agents/router.py; main.py	Intenciones sociales, perfil, resumen, auditoría y seguimiento contextual.
Persistencia	db.py	needs_review y migración idempotente de esquema.
Pruebas	backend/tests; frontend/src/**/*test.js	Regresiones de aislamiento, conversación, plan, perfil y restricciones.

Mejoras priorizadas

- Migraciones versionadas y restricciones de base de datos adicionales para reforzar actualizaciones y unicidad de progreso.
- Métricas de latencia, bloqueo, fallos del LLM y calidad de recuperación para operación basada en evidencia.
- Política de retención, exportación y eliminación de memoria y chat para completar el gobierno de datos personales.

Lectura final. La personalización de FitLife se apoya ahora en contexto autenticado, semántica explícita de restricciones y validación por capas. El LLM permanece como un complemento controlado, no como la autoridad de los datos estructurados.

Fin del documento.